

Demo RunPod Flash

Speech-to-Text Deployment trên Serverless GPU

Người thực hiện: Nguyễn Hải Thiện

Thời gian: 14:00, ngày 02/07/2026

GitHub: <https://github.com/adamwhite625/demo-runpod-stt>

Web Demo: <https://demo-runpod-stt.streamlit.app/>

Mở đầu

Báo cáo này trình bày kết quả demo hệ thống Speech-to-Text (STT) được triển khai trên nền tảng RunPod Flash, phục vụ cho bài toán Voice Bot Call Center. Mục tiêu gồm hai phần: (1) đo lường hiệu năng thực tế của mô hình faster-whisper trên GPU cloud, và (2) so sánh hai kiến trúc triển khai serverless (Flash vs Docker) để đưa ra khuyến nghị phù hợp cho từng giai đoạn phát triển sản phẩm.

1 Đánh giá Hiệu năng Speech-to-Text

1.1 Cấu hình hệ thống

Thành phần	Chi tiết
Mô hình	faster-whisper (large-v3-turbo)
GPU	NVIDIA RTX A5000 (24GB VRAM)
Compute Type	float16
Nền tảng	RunPod Flash Serverless
Workers	(0, 1) – hỗ trợ scale-to-zero
Idle Timeout	5 giây

Table 1: Cấu hình triển khai

1.2 Kịch bản 1: Hội thoại ngắn (15.5 giây)

File test: demo_stt_small.mp3 (61.6 KB, 15.5 giây). Nội dung mô phỏng cuộc gọi xác nhận đơn hàng giao hàng nhanh. Ngôn ngữ: Tiếng Việt – Confidence: 100%.

Upload Audio for Benchmark

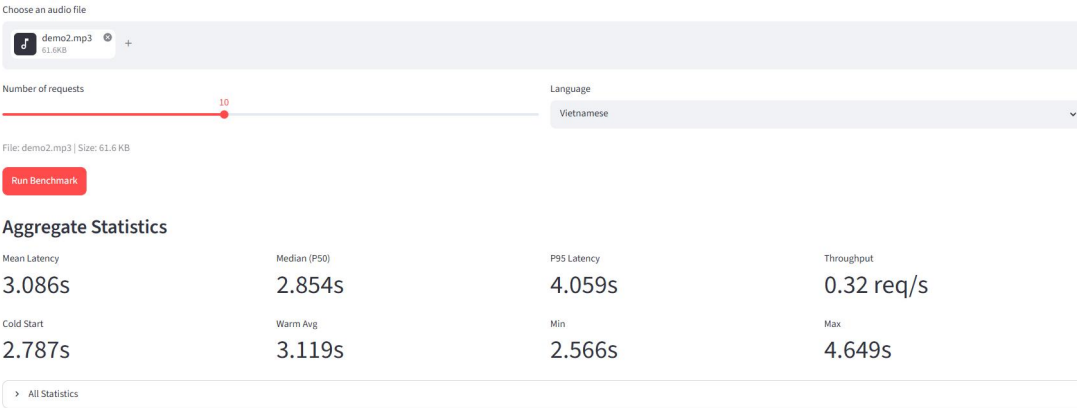


Figure 1: Kết quả transcription – file ngắn

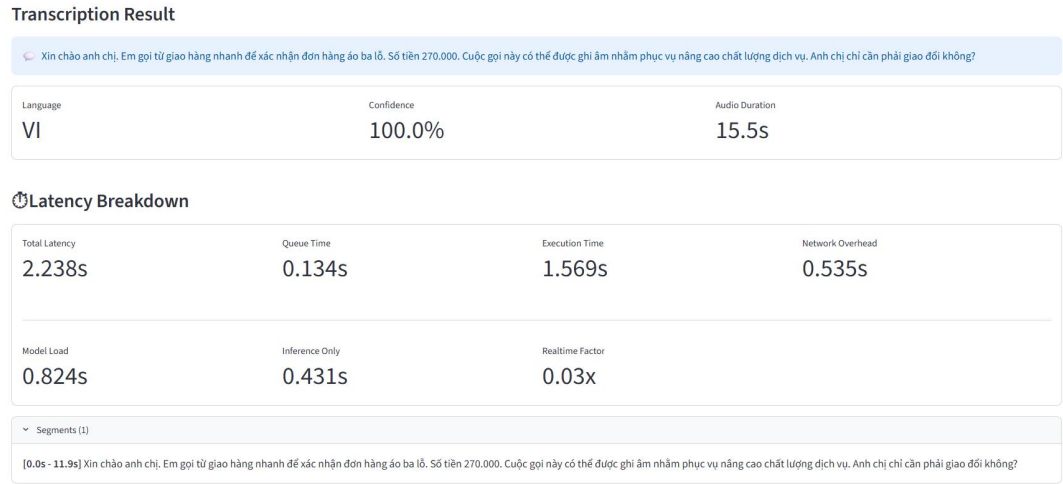


Figure 2: Kết quả benchmark – file ngắn

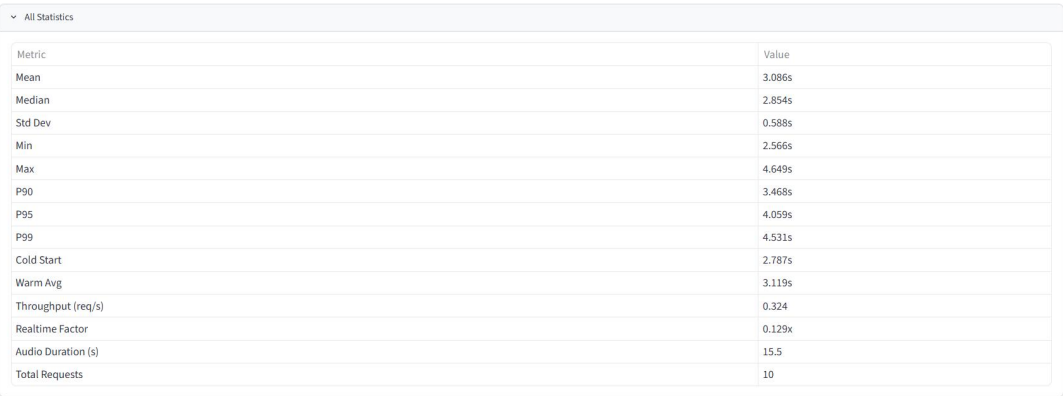


Figure 3: Thống kê chi tiết benchmark – file ngắn

1.2.1 Kết quả request đơn lẻ

Chỉ số	Giá trị
Total Latency (end-to-end)	2.238s
Queue Time	0.134s
Execution Time	1.569s
Network Overhead	0.535s
Model Load	0.824s
Inference Only	0.431s
Realtime Factor (RTF)	0.03x

Table 2: Latency breakdown – file ngắn, single request

RTF = 0.03x có nghĩa là thời gian inference thực tế (0.431s) chỉ bằng 3% độ dài audio (15.5s), tương đương tốc độ xử lý nhanh gấp khoảng 33 lần so với thời gian thực.

1.2.2 Benchmark 10 requests liên tục

Chỉ số	Giá trị
Mean Latency	3.086s
Median (P50)	2.854s
Std Dev	0.588s
P90	3.468s
P95	4.059s
P99	4.531s
Min / Max	2.566s / 4.649s
Cold Start	2.787s
Warm Average	3.119s
Throughput	0.324 req/s
Realtime Factor	0.129x

Table 3: Benchmark 10 requests – file ngắn

1.3 Kịch bản 2: Hội thoại dài (79.1 giây)

File test: demo_stt_large.mp3 (309.9 KB, 79.1 giây). Nội dung mô phỏng cuộc gọi xử lý khiếu nại giao hàng với nhiều lượt đối thoại. Ngôn ngữ: Tiếng Việt – Confidence: 100%.

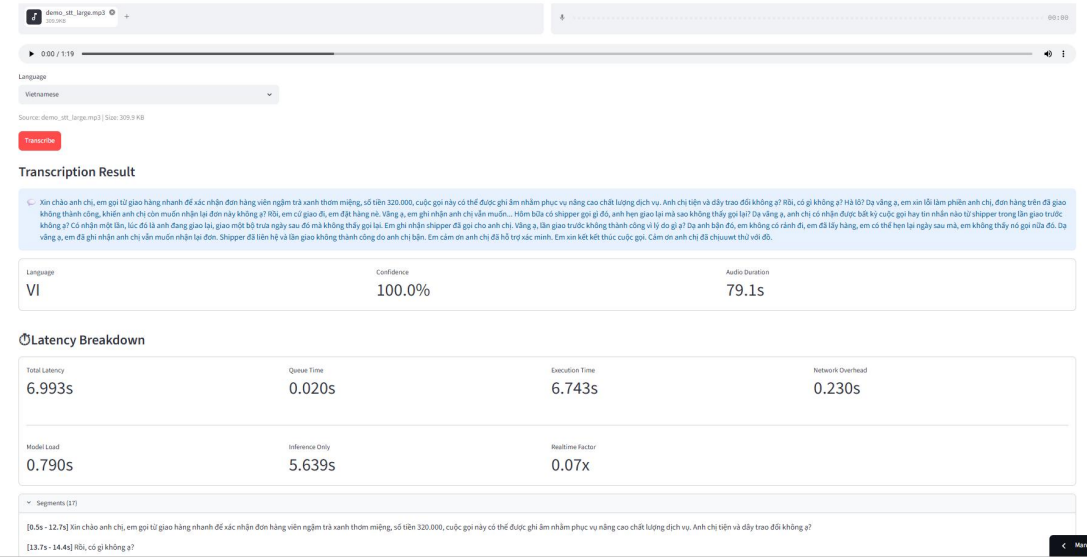


Figure 4: Kết quả transcription – file dài

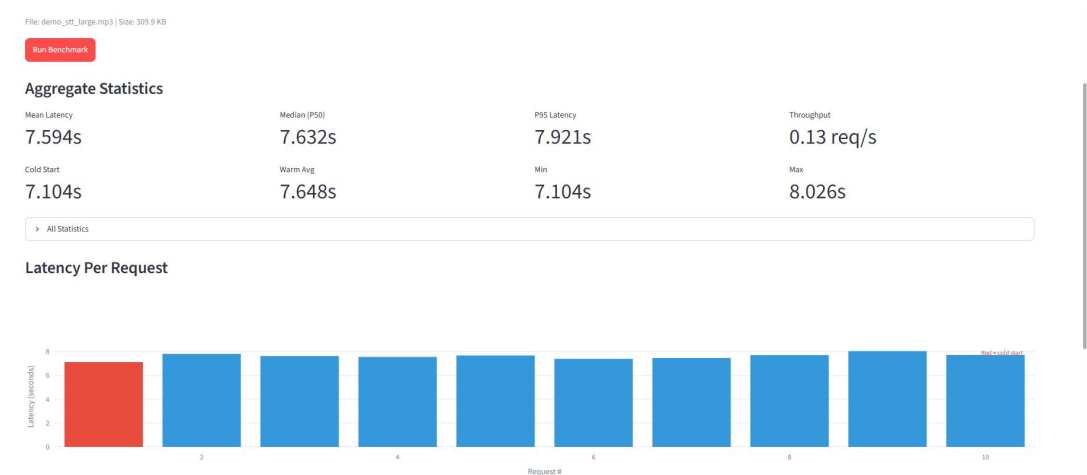


Figure 5: Kết quả benchmark – file dài

Raw Per-Request Data									
	request_num	total_latency	queue_time	execution_time	network_overhead	model_load_time	inference_time	audio_duration	status
	0	1	7.104	0.038	6.814	0.252	0.716	5.796	79.1 COMPLETED
	1	2	7.793	0.84	6.737	0.216	0.735	5.659	79.1 COMPLETED
	2	3	7.607	0.842	6.584	0.181	0.59	5.554	79.1 COMPLETED
	3	4	7.536	0.85	6.504	0.182	0.628	5.558	79.1 COMPLETED
	4	5	7.857	0.84	6.623	0.194	0.603	5.693	79.1 COMPLETED
	5	6	7.376	0.868	6.315	0.193	0.672	5.33	79.1 COMPLETED
	6	7	7.448	0.871	6.412	0.165	0.73	5.302	79.1 COMPLETED
	7	8	7.688	0.835	6.656	0.197	0.714	5.663	79.1 COMPLETED
	8	9	8.026	0.706	6.985	0.335	0.657	6.059	79.1 COMPLETED
	9	10	7.701	0.842	6.665	0.194	0.684	5.66	79.1 COMPLETED

Figure 6: Dữ liệu thô từng request – file dài



Figure 7: Biểu đồ phân bổ thời gian từng request – file dài

1.3.1 Kết quả request đơn lẻ

Chỉ số	Giá trị
Total Latency (end-to-end)	6.993s
Queue Time	0.020s
Execution Time	6.743s
Network Overhead	0.230s
Model Load	0.790s
Inference Only	5.639s
Realtime Factor (RTF)	0.07x

Table 4: Latency breakdown – file dài, single request

Dù file audio dài gấp 5 lần so với kịch bản 1, RTF vẫn giữ ở mức 0.07x – cho thấy GPU A5000 không gặp tình trạng nghẽn khi xử lý chuỗi dữ liệu dài.

1.3.2 Benchmark 10 requests liên tục

Chỉ số	Giá trị
Mean Latency	7.594s
Median (P50)	7.632s
P95	7.921s
Min / Max	7.104s / 8.026s
Cold Start	7.104s
Warm Average	7.648s
Throughput	0.13 req/s

Table 5: Benchmark 10 requests – file dài

1.3.3 Dữ liệu chi tiết từng request

#	Total (s)	Queue (s)	Exec (s)	Network (s)	Load (s)	Infer (s)
1	7.104	0.038	6.814	0.252	0.716	5.796
2	7.793	0.840	6.737	0.216	0.735	5.659
3	7.607	0.842	6.584	0.181	0.590	5.554
4	7.536	0.850	6.504	0.182	0.628	5.558
5	7.657	0.840	6.623	0.194	0.603	5.693
6	7.376	0.868	6.315	0.193	0.672	5.330
7	7.448	0.871	6.412	0.165	0.730	5.392
8	7.688	0.835	6.656	0.197	0.714	5.663
9	8.026	0.706	6.985	0.335	0.657	6.059
10	7.701	0.842	6.665	0.194	0.684	5.660

Table 6: Dữ liệu chi tiết 10 requests – file dài (79.1s audio)

1.4 Nhận xét tổng hợp

- RTF luôn dưới 0.1x ở cả hai kịch bản, tốc độ xử lý nhanh hơn thời gian thực ít nhất 10 lần.
- Độ lệch chuẩn thấp cho thấy thời gian phản hồi ổn định giữa các request, không suy giảm hiệu năng khi chạy liên tục.
- Cold Start chỉ ảnh hưởng ở request đầu tiên. Các request tiếp theo (warm) gần như không chênh lệch.
- Execution Time chiếm phần lớn tổng latency, Queue Time và Network Overhead rất nhỏ – hạ tầng mạng của RunPod hoạt động ổn định.

2 So sánh Kiến trúc Triển khai: RunPod Flash vs Docker Serverless

Khi triển khai mô hình AI lên RunPod, có hai lựa chọn chính: sử dụng RunPod Flash (viết hàm Python kèm decorator `@Endpoint`) hoặc tự xây dựng Docker Image và deploy qua cơ chế Serverless truyền thống. Bảng dưới đây so sánh hai phương pháp dựa trên các tiêu chí vận hành thực tế.

Tiêu chí	RunPod Flash	Docker Serverless
Tốc độ deploy	Nhanh (vài giây). Chỉ cần gõ lệnh <code>flash deploy</code> .	Chậm hơn (vài phút). Cần build image, push lên registry, pull về server.
Quản lý cấu hình	Viết trực tiếp trong code Python qua decorator <code>@Endpoint</code> .	Cấu hình trên Web UI hoặc qua API/Terraform riêng.
Quản lý dependencies	Khai báo qua tham số <code>dependencies</code> , hệ thống tự cài đặt.	Toàn quyền kiểm soát thông qua <code>Dockerfile</code> .
Tối ưu Cold Start	Cần tải model weights từ nguồn bên ngoài (HuggingFace, S3) mỗi lần worker khởi tạo lại.	Có thể đóng gói sẵn model weights vào Docker Image, giảm thời gian Cold Start.
Kiểm soát version	Phụ thuộc vào base image nội bộ của RunPod.	Toàn quyền kiểm soát version thư viện, OS, CUDA driver.

Table 7: So sánh RunPod Flash và Docker Serverless

2.1 Kết luận

Dựa trên các đặc tính trên, đề xuất chia chiến lược triển khai thành hai giai đoạn:

Giai đoạn R&D và Testing – nên dùng RunPod Flash. Trong quá trình phát triển, team thường xuyên thử nghiệm nhiều model khác nhau (Whisper large-v2, v3, turbo) và thay đổi dependencies liên tục. RunPod Flash giúp giảm thiểu việc quản lý Docker image, tránh phải xử lý xung đột version giữa các lần thử nghiệm, cho phép team tập trung vào việc đánh giá model và demo kết quả nhanh.

Giai đoạn Production – nên dùng Docker Serverless. Khi đã chốt được model và đưa vào phục vụ khách hàng thực tế, hệ thống cần tính ổn định cao và khả năng bảo trì lâu dài. Docker Serverless cho phép đóng gói cứng toàn bộ version thư viện và model weights thành một image duy nhất, giúp kiểm soát version chặt chẽ, đồng thời loại bỏ thời gian tải model qua mạng – giảm đáng kể độ trễ Cold Start khi vận hành thực tế.

3 Kiến trúc Multi-Model trên Serverless GPU (STT & TTS)

Dựa trên yêu cầu của hệ thống Voice Bot thực tế, chúng tôi đã nâng cấp kiến trúc RunPod Flash để xử lý đồng thời hai tác vụ: Nhận dạng giọng nói (Whisper STT) và Sinh giọng nói nhái giọng (XTTS-v2 Voice Cloning) trên **cùng một máy ảo GPU A5000**.

3.1 Tối ưu không gian lưu trữ (Network Volume)

Thay vì tải Model Weights từ internet mỗi khi có Cold Start (lên tới 3.7GB cho cả 2 model), toàn bộ weights được lưu trữ cố định trên **RunPod Network Volume**.

- **Giảm thiểu Cold Start:** Việc nạp model từ ổ cứng mạng SSD vào GPU VRAM (tốc độ 300MB/s) giúp triệt tiêu hoàn toàn độ trễ tải mạng. Thời gian Cold Start cho cả 2 model giảm từ hơn 1 phút xuống chỉ còn **15-25 giây**.
- **Tối ưu gói triển khai:** Kích thước package code đẩy lên RunPod chỉ còn vài chục KB, giúp việc deploy diễn ra tức thời.

3.2 Xử lý luồng (Routing) & Batch Processing

Để tiết kiệm chi phí, thay vì tách STT và TTS thành 2 API độc lập (khiến hệ thống phải duy trì 2 GPU riêng biệt), chúng tôi cấu hình **Unified Endpoint** nhận tham số `action="stt"` hoặc `action="tts"`. Hệ thống sử dụng Global Caching để giữ cả 2 model thường trực trong 24GB VRAM của A5000 (Whisper: ~3GB, XTTS-v2: ~2.1GB).

Đối với quá trình xử lý Batch cho STT, hệ thống áp dụng kỹ thuật **Server-Side Concurrency (GPU-level batching)** đạt chuẩn Production:

- Khởi tạo WhisperModel với `num_workers=4`.
- Sử dụng `ThreadPoolExecutor` kết hợp `CTranslate2` để đẩy nhiều file âm thanh vào GPU xử lý đồng thời trên các luồng CUDA độc lập.

- **Kết quả:** Cho phép xử lý cùng lúc nhiều phiên (sessions) hội thoại song song với hiệu năng vượt trội.